



Mensageria

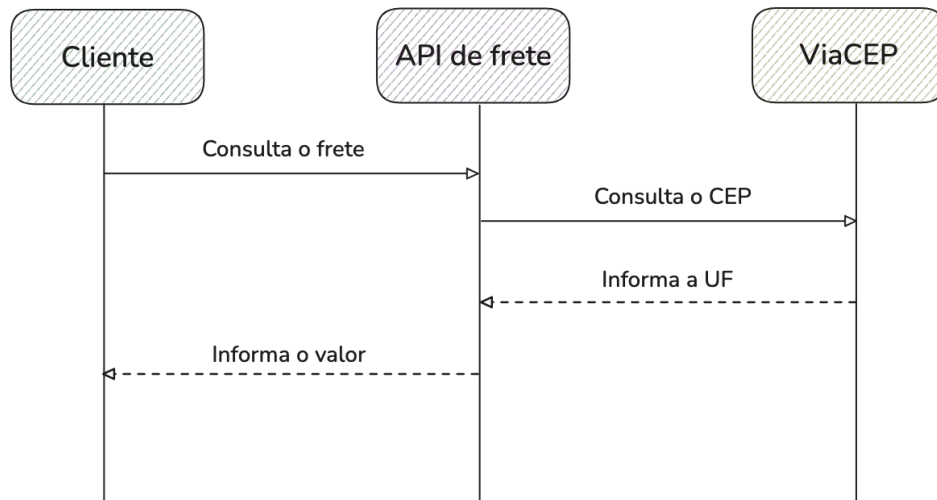
Integração de Sistemas - Aula #06

Prof. Dr. Carlos Melo

 carlos.melo@upe.br

Continuando...

- Nossas integrações seguiram um mesmo modelo:
 - Um programa envia uma requisição e outro processa
 - Quem faz a chamada espera uma resposta.
- Na API de frete:



Confirmação do pedido

- Agora o sistema também deve receber pedidos e quando o cliente confirma uma compra:
 - A API registra o pedido
 - O serviço de notificações envia uma confirmação
 - O cliente recebe o resultado da operação.
- Podemos fazer tudo durante a mesma requisição:



Primeira Implementação

- A API registra o pedido e chama o serviço de notificações
- A chamada termina somente depois que ele responde.

```
@app.post("/pedidos", status_code=201)
def criar_pedido(pedido: Pedido):
    pedido_id = len(pedidos) + 1
    pedidos.append({
        "id": pedido_id,
        **pedido.model_dump()
    })

    resposta = httpx.post(
        "http://notificacoes:8000/confirmacoes",
        json={
            "pedido_id": pedido_id,
            "email": pedido.email
        },
        timeout=3
    )
    resposta.raise_for_status()

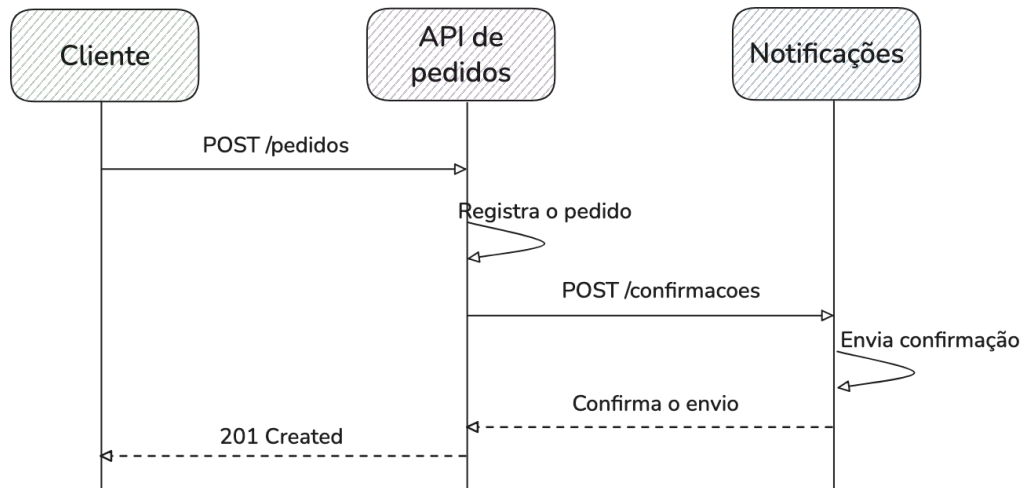
    return {
        "id": pedido_id,
        "status": "confirmado"
    }
```

Primeiro teste

- Na pasta do projeto rode:
 - `docker compose up --build`
- Abra <http://localhost:8000/docs>
- E faça um POST em /pedidos
 - `{"cliente": "Ana", "email": "ana@email.com", "produto": "Notebook"}`
- A resposta esperada é
 - `{"id": 1, "status": "confirmado"}`
- Nos logs, procure a confirmação enviada pelo segundo serviço:
 - `notificacoes | Confirmação do pedido 1 enviada para ana@email.com`

Processamento síncrono do pedido

- A API somente responde quando o serviço de notificações termina.



Teste de indisponibilidade

- Pare somente o serviço de notificações:
 - `docker compose stop notificacoes`
- No Swagger, execute novamente POST /pedidos com outro produto
 - `{"cliente": "Ana", "email": "ana@email.com", "produto": "Teclado"}`
- Depois do erro execute um GET /pedidos
 - Qual foi a resposta de POST /pedidos?
 - O pedido do teclado aparece na listagem?
 - O cliente sabe se a compra foi registrada?

Pedido salvo e resposta errada

- A execução parou nesta linha:

```
resposta = httpx.post(  
    "http://notificacoes:8000/confirmacoes",  
    json={  
        "pedido_id": pedido_id,  
        "email": pedido.email  
    },  
    timeout=3  
)
```

- Entretanto, o pedido já tinha sido registrado e se o cliente tenta refazer a compra vai acabar gastando duas vezes.

```
pedidos.append({  
    "id": pedido_id,  
    **pedido.model_dump()  
})
```

Acoplamento temporal

- Na comunicação síncrona, quem chama permanece esperando por quem foi chamado.
- Para concluir POST /pedidos:
 - A API de pedidos precisa estar disponível
 - O serviço de notificações também precisa responder naquele momento
 - A lentidão das notificações aumenta o tempo da requisição
 - A falha das notificações chega ao cliente como falha do pedido.
- Mesmo executando em contêineres separados, os serviços continuam acoplados temporalmente.

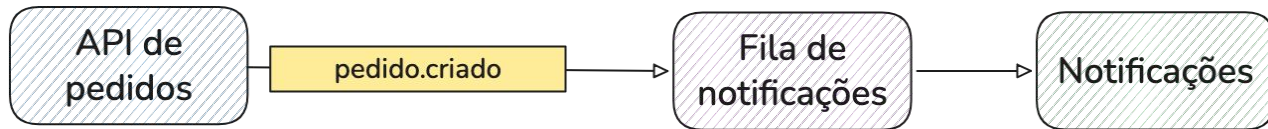


Antes e depois da resposta

- Para aceitar o pedido, a API precisa:
 - Validar os dados
 - Registrar o pedido
- O envio da confirmação pode continuar depois da resposta.
 - Validar → Registrar pedido → Responder ao cliente → Enviar confirmação
- Precisamos guardar essa tarefa até que o serviço de notificações consiga executá-la.
 - Onde podemos guardar essa solicitação de envio sem manter o cliente esperando?

Fila de mensagens

- A API pode colocar uma mensagem em uma fila depois de registrar o pedido
 - A API coloca a mensagem na fila.
 - O serviço de notificações retira a mensagem quando estiver disponível.
 - A fila mantém o trabalho pendente durante a indisponibilidade do consumidor.
 - A API já pode responder ao cliente sem esperar pelo envio da confirmação.



Mensageria assíncrona

- Os componentes da nova integração recebem nomes específicos:
 - Produtor: cria e envia a mensagem
 - Broker: recebe e mantém a mensagem
 - Consumidor: retira e processa a mensagem
- Esse modelo permite que produtor e consumidor trabalhem em momentos diferentes.



RabbitMQ

- RabbitMQ será o broker da nossa aplicação.
- As aplicações utilizarão a porta 5672
- A interface de administração do coelho estará disponível na porta 15672
 - Aplicações conectadas
 - Filas existentes
 - Mensagens aguardando processamento

RabbitMQ no Compose

- Acrescente o serviço ao `compose.yaml`:
- Suba o novo serviço:
 - `docker compose up -d rabbitmq`
 - Confira se o contêiner está *up*



```
services:
  rabbitmq:
    image: rabbitmq:4-management
    environment:
      RABBITMQ_DEFAULT_USER: aula
      RABBITMQ_DEFAULT_PASS: aula123
    ports:
      - "5672:5672"
      - "15672:15672"
```

Interface de administração

- Abra no navegador `http://localhost:15672`
- Entre com usuário e senha previamente definidos
- Na interface:
 - Abra Connections
 - Abra Queues and Streams
 - Confirme que nenhuma aplicação está conectada
 - Confirme que ainda não existe uma fila criada pela aplicação
- Em tese, o RabbitMQ está funcionando, mas ninguém publicou uma mensagem.

Caminho da mensagem

- A API deixará de chamar o serviço de notificações por HTTP
- Ao criar um pedido, a API deverá:
 - Montar uma mensagem
 - Conectar-se ao RabbitMQ
 - Garantir que a fila existe
 - Publicar a mensagem
 - Fechar a conexão
- Sem um consumidor pronto a mensagem ficará guardada na fila



Publicação no RabbitMQ

- Vamos usar a biblioteca pika e adicioná-la ao requirements.txt
- Na [main.py](#) crie a função publicar()

```
import json
import pika

def publicar(evento: dict):
    parametros = pika.URLParameters(
        "amqp://aula:aula123@rabbitmq:5672/"
    )

    conexao = pika.BlockingConnection(parametros)
    canal = conexao.channel()

    canal.queue_declare(
        queue="pedidos",
        durable=True
    )

    canal.basic_publish(
        exchange="",
        routing_key="pedidos",
        body=json.dumps(evento).encode("utf-8")
    )

    conexao.close()
```

Publicação do pedido

- Remova a chamada HTTP ao serviço de notificações
- Depois de registrar o pedido, monte e publique o evento:

```
from uuid import uuid4

@app.post("/pedidos", status_code=202)
def criar_pedido(pedido: Pedido):
    pedido_id = len(pedidos) + 1

    pedidos.append({
        "id": pedido_id,
        **pedido.model_dump()
    })

    evento = {
        "mensagem_id": str(uuid4()),
        "tipo": "pedido.criado",
        "versao": 1,
        "dados": {
            "pedido_id": pedido_id,
            "email": pedido.email
        }
    }

    publicar(evento)

    return {
        "id": pedido_id,
        "status": "recebido"
    }
```

Mensagem aguardando

- Reconstrua a API
 - `docker compose up -d --build pedidos rabbitmq`
- Abra novamente `http://localhost:8000/docs`
- Execute POST `/pedidos` e confirme a resposta
 - `{"id": 1, "status": "recebido"}`
- Na interface do RabbitMQ:
 - Abra Queues and Stream
 - Selecione a fila pedidos
 - Localize o contador Ready (Tem que ter valor 1)
 - Localize o contador Consumer (Tem que ter valor 0)

Estrutura da mensagem

- A API publicou um dicionário serializado como JSON:

```
{
  "mensagem_id": "9f6de6d5-5e7a-4b8a-9de6-87114366bd37",
  "tipo": "pedido.criado",
  "versao": 1,
  "dados": {
    "pedido_id": 1,
    "email": "ana@email.com"
  }
}
```

Campo	Significado
<code>mensagem_id</code>	Distingue uma mensagem das demais
<code>tipo</code>	Informa o fato representado
<code>versao</code>	Identifica o formato da mensagem
<code>dados</code>	Contém as informações do pedido

- O RabbitMQ recebe bytes. `json.dumps()` e `encode()` transformam o dicionário antes do envio.

Contrato usado pelo consumidor

- O consumidor precisa localizar:
 - O tipo do evento
 - O identificador do pedido
 - O endereço para confirmação.
- Se o produtor alterar esses campos, o consumidor também precisará mudar.

```
{
  "mensagem_id": "9f6de6d5-5e7a-4b8a-9de6-87114366bd37",
  "tipo": "pedido.criado",
  "versao": 1,
  "dados": {
    "pedido_id": 1,
    "email": "ana@email.com"
  }
}
```

Consumidor de notificações

- Vamos adicionar a pika ao requirements.txt do serviço notificacoes

```
# notificacoes/main.py
import json
import pika

parametros = pika.URLParameters("amqp://aula:aula123@rabbitmq:5672/")

conexao = pika.BlockingConnection(parametros)
canal = conexao.channel()

canal.queue_declare(
    queue="pedidos",
    durable=True
)

def confirmar(ch, metodo, propriedades, corpo):
    evento = json.loads(corpo.decode("utf-8"))
    dados = evento["dados"]

    print(
        f"Pedido {dados['pedido_id']}: "
        f"confirmação enviada para {dados['email']}",
        flush=True
    )

canal.basic_consume(
    queue="pedidos",
    on_message_callback=confirmar,
    auto_ack=True
)

print("Aguardando pedidos...", flush=True)
canal.start_consuming()
```

Primeiro consumo

- Inicie o consumidor
 - `docker compose up -d --build notificacoes`
 - `docker compose logs -f notificacoes`
- Resultado esperado:
 - Aguardando pedidos...
 - Pedido 1: confirmação enviada para ana@email.com
- No RabbitMQ:
 - Ready: 0
 - Consumers: 1
- A mensagem publicada anteriormente permaneceu na fila e chegou ao consumidor quando ele iniciou.
- Ctrl+C encerra somente a visualização dos logs.

Primeiro consumo

- Processamento assíncrono
 - O cliente envia POST /pedidos
 - A API registra o pedido
 - A API publica pedido.criado
 - A API responde 202 Accepted
 - O RabbitMQ entrega a mensagem
 - O serviço de notificações processa o pedido
- A resposta da API não espera o envio da confirmação.

Consumidor fora do ar

- Interrompa o consumidor:
 - `docker compose stop notificacoes`
- Envie três pedidos pelo Swagger
- Antes de continuar, observe:
 - API: três respostas 202
 - Ready: 3
 - Consumers: 0
- Reinicie o consumidor
 - `docker compose start notificacoes`
 - `docker compose logs -f notificacoes`
- As três confirmações aparecem nos *logs*
- O total em Ready é zero e temos um Consumer

O consumidor parado não perdeu mensagens, mas o que acontece se ele cair depois de receber uma???

Falha durante o processamento

- Adicione uma pausa ao consumidor:

```
import time

def confirmar(ch, metodo, propriedades, corpo):
    evento = json.loads(corpo.decode("utf-8"))
    dados = evento["dados"]

    print(f"Processando pedido {dados['pedido_id']}...", flush=True)
    time.sleep(20)

    print(f"Confirmação enviada para {dados['email']}", flush=True)
```

- Mantenha `auto_ack=True`
- Execute o consumidor, envie um pedido e aguarde a mensagem Processando pedido....
- Em outro terminal mate o consumidor
 - `docker compose kill notificacoes`

Falha durante o processamento

- Resultado:
 - Confirmação enviada: não
 - Ready: 0
 - Consumers: 0
- A mensagem desaparece mesmo sem a conclusão do processamento

Confirmação de processamento

- O RabbitMQ pode aguardar uma confirmação do consumidor
 - `auto_ack=True` significa que RabbitMQ remove a mensagem assim que a entrega
 - `auto_ack=False` significa que a mensagem permanece sem confirmação até o consumidor enviar um ack
- Com confirmação manual, uma mensagem pode assumir estes estados:
 - Ready: mensagem aguardando entrega
 - Unacked: mensagem entregue, mas ainda não confirmada
 - Concluída: consumidor enviou o ack
- Se a conexão terminar enquanto a mensagem estiver em Unacked, o RabbitMQ poderá colocá-la novamente em Ready.

Confirmação manual

- Altere o consumidor:

```
def confirmar(ch, metodo, propriedades, corpo):  
    evento = json.loads(corpo.decode("utf-8"))  
    dados = evento["dados"]  
  
    print(f"Processando pedido {dados['pedido_id']}...", flush=True)  
    time.sleep(20)  
    print(f"Confirmação enviada para {dados['email']}", flush=True)  
  
    ch.basic_ack(delivery_tag=metodo.delivery_tag)
```

- Também altere o consumo, assim o consumidor envia o ack somente depois de concluir o trabalho. Mas o que acontece se o contêiner cair durante o `time.sleep`?

```
canal.basic_consume(  
    queue="pedidos",  
    on_message_callback=confirmar,  
    auto_ack=False  
)
```

Reentrega após uma falha

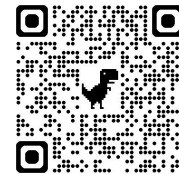
- Reconstrua o consumidor:
 - `docker compose up -d --build notificacoes`
 - `docker compose logs -f notificacoes`
- Envie um novo pedido
- Enquanto o consumidor estiver processando, observe:
 - Ready: 0
 - Unacked: 1
 - Consumers: 1
- Interrompa o contêiner:
 - `docker compose kill notificacoes`
- A mensagem retorna à fila:
 - Ready: 1
 - Unacked: 0
 - Consumers: 0

Reentrega após uma falha

- Reinicie o consumidor:
 - `docker compose up -d notificacoes`
 - `docker compose logs -f notificacoes`
- O mesmo pedido aparece novamente nos *logs*.
- Depois do ack:
 - Ready: 0
 - Unacked: 0

Entrega pelo menos uma vez

- A confirmação manual evita a perda observada no primeiro teste, mas permite reentregas
- Exemplo:
 - O consumidor envia a confirmação ao cliente
 - O processo termina antes de enviar o ack
 - O RabbitMQ devolve a mensagem para a fila
 - Outro consumidor processa a mesma mensagem
 - O cliente recebe uma segunda confirmação
- Esse comportamento recebe o nome de entrega pelo menos uma vez:
 - Uma mensagem pode ser processada uma ou mais vezes
- O campo `id_evento` permite reconhecer uma mensagem já processada.
- Como impedir que uma reentrega produza duas confirmações (*ex.: duas compras*)?
 - Fica o desafio.



Prof. Dr. Carlos Melo
<https://calendly.com/prof-casm>

 carlos.melo@upe.br